

Bug #7 – Balance Race Condition / Incorrect Balance Update

1. Issue Summary

The system currently has multiple balance-related operations where the user's balance is read, calculated, and updated without sufficient concurrency protection.

When two or more balance-related requests are processed at the same time for the same user, both requests can read the same old balance and then update it independently.

This can result in:

- Incorrect user balance
- Negative balance
- Lost balance updates
- Incorrect ledger `balance_after` values
- Trade/deposit/withdrawal amounts being overwritten
- Inconsistent financial records

The identified issue is classified as a **functional/correctness bug** because it can cause the trading system to display or maintain an incorrect financial balance.

2. Example of the Problem

Assume a user has:

Current Balance: ₹1,000

Two withdrawal/transfer requests arrive at almost the same time:

- Request A → ₹800
- Request B → ₹800

Both requests can read:

`Balance = ₹1,000`

Both may pass the balance validation because:

`₹1,000 >= ₹800`

Without proper concurrency control, both operations may proceed.

This creates an invalid financial state because the user only had ₹1,000 available while the two requests together require ₹1,600.

The audit identified this type of concurrent balance validation/update as a race-condition problem.

3. Lost Update Problem

Another identified pattern is:

1. Read current balance
2. Calculate a new balance in application memory
3. Save the calculated balance back to the database

For example:

Current Balance = ₹10,000

One request calculates:

₹10,000 + ₹5,000 = ₹15,000

At the same time, another request calculates:

₹10,000 - ₹2,000 = ₹8,000

If both requests started from the same old balance, the later absolute update can overwrite the earlier update.

This can cause a legitimate balance change to disappear from the final database balance.

4. Identified Affected Areas

The audit identified the following backend files/functions as connected to the balance race-condition issue:

4.1 portfolioController.js

Function: `internalTransfer`

The sender's balance is checked without an appropriate row lock.

Concurrent transfers can therefore operate against the same balance state and potentially produce an incorrect balance.

4.2 fundController.js

Functions:

- addFund
- updateFund
- deleteFund

The audit identified an absolute balance update pattern such as:

```
UPDATE users SET balance = ?
```

This can overwrite another concurrent balance change if the calculated value was based on an older balance state.

4.3 requestController.js

Function: updateStatus

This handles withdrawal/deposit status changes.

The audit identified a balance read without row locking followed by the use of the balance value for ledger `balance_after`.

Under concurrent operations, this can result in incorrect balance and ledger values.

4.4 WeeklySettlementService.js

Function: processUserSettlement

The audit identified an absolute calculated balance update.

The calculated balance can potentially overwrite balance changes that occur while settlement calculations are being performed.

4.5 TradeService.js

Function: `closeTrade`

Trade closing updates the user's balance.

The audit identified that the user balance row does not have appropriate transaction locking during this operation, which can allow concurrent balance operations to interfere with each other.

4.6 `dbService.js`

Function: `addFund`

The audit identified a read → calculate → write pattern without sufficient transaction/row-lock protection.

This can also result in the ledger's `balance_after` becoming out of sync with the actual account balance.

4.7 `aiExecutor.js`

Functions:

- `executeAddFund`
- `executeDeductFund`

These operations perform balance calculations without sufficient row-lock protection.

The audit identified a risk of the ledger `balance_after` becoming inconsistent with the actual balance.

5. Required Fix

The primary objective is to make all critical balance operations **concurrency-safe**.

A. Transaction Protection

Balance operations should be handled inside appropriate database transactions.

The logical operation should remain consistent:

Balance validation

→ Balance update

→ Ledger entry

→ Commit

If an operation fails, the related changes should be rolled back.

B. Row Locking

Where the application needs to check the current balance before deducting funds, the balance row should be locked during the transaction.

Recommended pattern:

```
SELECT balance FROM users WHERE id = ? FOR UPDATE
```

This ensures that another transaction cannot modify the same user balance simultaneously until the current transaction completes.

C. Prevent Negative/Over-Deduction

Balance deduction should also enforce the available-balance condition at the database level.

Recommended pattern:

```
UPDATE users SET balance = balance - ? WHERE id = ? AND balance >= ?
```

If no row is affected, the operation should fail and the transaction should be rolled back.

D. Accurate Ledger Balance

The ledger `balance_after` value must represent the actual balance resulting from the completed transaction.

The balance update and ledger insertion should therefore be coordinated within the same transaction/locking flow.

6. Important Implementation Note

The solution should **not blindly replace** every:

```
UPDATE users SET balance = ?
```

with:

```
UPDATE users SET balance = balance + ?
```

An absolute balance update is not automatically incorrect if it is intentionally calculating a final balance inside a properly protected transaction.

The actual issue is the unsafe:

```
Read → Calculate → Write
```

pattern when it occurs without appropriate concurrency protection.

Therefore, each affected function should be reviewed individually before changing its balance logic.

7. Testing Required After Fix

After implementation, concurrent requests must be tested.

Test Case 1 – Concurrent Deduction

Initial balance:

```
₹10,000
```

Simultaneous requests:

- Deduct ₹6,000
- Deduct ₹4,000

Expected:

```
Final Balance = ₹0
```

No amount should be lost or overwritten.

Test Case 2 – Over-Deduction Attempt

Initial balance:

₹10,000

Simultaneous requests:

- Deduct ₹7,000
- Deduct ₹5,000

Total requested:

₹12,000

Expected:

- Total successful deduction must not exceed ₹10,000
 - Balance must never become negative
 - One transaction must fail if sufficient balance is no longer available
-

Test Case 3 – Concurrent Credit/Debit

Initial balance:

₹10,000

At the same time:

- Credit ₹5,000
- Debit ₹2,000

Expected final balance:

₹13,000

The final database value must correctly include both operations.

Test Case 4 – Ledger Verification

For every successful balance transaction verify:

Opening Balance + Credits - Debits = Final Balance

And verify that the ledger `balance_after` matches the actual balance after each transaction.

8. Scope – What This Task Includes

This task includes:

- User balance read protection
- Balance deduction protection
- Balance credit protection
- Concurrent balance updates
- Transaction handling
- Row locking where required
- Negative/over-deduction prevention
- Ledger `balance_after` consistency

The overall objective is to make user balance operations **thread-safe and race-condition resistant**.

9. Scope – What This Task Does NOT Include

The following issues are outside Bug #7:

- Market/live execution price issue
- Trade closing P&L calculation
- Trade ownership validation
- Negative transfer amount input validation
- Pending-order cancellation/refund logic
- Trade double-close/idempotency
- Pending Order / Target-SL double execution
- Fractional quantity database schema
- Missing/stale market price handling
- MCX brokerage calculation
- Lot-size mapping issues

These should remain with their respective bug/feature owners to avoid scope overlap.

10. Final Conclusion

The current issue is a **functional balance correctness problem**.

Under concurrent requests, the existing balance handling can potentially allow multiple operations to work from the same stale balance or overwrite another balance update.

The fix should ensure that:

One user's balance cannot be incorrectly modified by competing transactions.

The implementation should use appropriate:

- Database transactions
- Row-level locking
- Atomic balance operations
- Balance validation
- Negative-balance protection
- Ledger consistency

After implementation, concurrent transaction tests must confirm that the final balance always matches the actual successful financial operations.

Current Status: Issue Identified – Fix Pending

Code Changes: Not yet performed

Priority: High – Financial/Trading Correctness